

UNIT-II

Chapter-1
Searching**1.1 Introduction**

Let DATA be a collection of data elements in memory, and suppose a specific ITEM of information is given. Searching refers to the operation of finding the location LOC of ITEM in DATA, or printing some message that ITEM does not appear there. The search is said to be successful if ITEM does appear in DATA and unsuccessful otherwise.

Two types of searching Techniques are:

1. Linear Search/Sequential Search
2. Binary Search

1.2 Linear Search/Sequential Search

Suppose DATA is a linear array with n elements. Given no other information about DATA, the most intuitive way to search for a given ITEM in DATA is to compare ITEM with each element of DATA one by one. That is, first we test whether $DATA[I] = ITEM$, and then we test whether $DATA[2] = ITEM$, and so on. This method, which traverses DATA sequentially to locate ITEM, is called linear search or sequential search. To simplify the matter, we first assign ITEM to $DATA[N + 1]$, the position following the last element of DATA. Then the outcome $LOC = N + 1$ where LOC denotes the location where ITEM first occurs in DATA, signifies the search is unsuccessful.

The purpose of this initial assignment is to avoid repeatedly testing whether or not we have reached the end of the array DATA. This way, the search must eventually "succeed." A formal presentation of linear search is shown in Algorithm 4.5. Observe that Step 1 guarantees that the loop in Step 3 must terminate. Without Step 1 (see Algorithm 2.4), the Repeat statement in Step 3 must be replaced by the following statement, which involves two comparisons, not one: Repeat while $LOC \leq N$ and $DATA[LOC] \neq ITEM$: On the other hand, in order to use Step 1, one must guarantee that there is an unused memory location at the end of the array DATA; otherwise, one must use the linear search algorithm.

Algorithm 4.5: (Linear Search) LINEAR(DATA, N, ITEM, LOC)

Here DATA is a linear array with N elements, and ITEM is a given item of information. This algorithm finds the location LOC of ITEM in DATA, or sets $LOC := 0$ if the search is unsuccessful.

1. [Insert ITEM at the end of DATA.] Set $DATA[N + 1] := ITEM$.
2. [Initialize counter.] Set $LOC := 1$.
3. [Search for ITEM.]
Repeat while $DATA[LOC] \neq ITEM$:
Set $LOC := LOC + 1$.
[End of loop.]
4. [Successful?] If $LOC = N + 1$, then: Set $LOC := 0$.
5. Exit.

Example 4.9

Consider the array NAME in Fig. 4.6(a), where $n = 6$.

- (a) Suppose we want to know whether Paula appears in the array and, if so, where. Our algorithm temporarily places Paula at the end of the array, as pictured in Fig. 4.6(b), by setting $NAME[7] = \text{Paula}$. Then the algorithm searches the array from top to bottom. Since Paula first appears in $NAME[N + 1]$, Paula is not in the original array.
- (b) Suppose we want to know whether Susan appears in the array and, if so, where. Our algorithm temporarily places Susan at the end of the array, as pictured in Fig. 4.6(c), by setting $NAME[7] = \text{Susan}$. Then the algorithm searches the array from top to bottom. Since Susan first appears in $NAME[4]$ (where $4 \leq n$), we know that Susan is in the original array.

NAME		NAME		NAME	
1	Mary	1	Mary	1	Mary
2	Jane	2	Jane	2	Jane
3	Diane	3	Diane	3	Diane
4	Susan	4	Susan	4	Susan
5	Karen	5	Karen	5	Karen
6	Edith	6	Edith	6	Edith
7		7	Paula	7	Susan
8		8		8	

(a)

(b)

(c)

Fig. 4.6

1.3 Binary Search

Suppose DATA is an array which is sorted in increasing numerical order or, equivalently, alphabetically. Then there is an extremely efficient searching algorithm, called binary search, which can be used to find the location LOC of a given ITEM of information in DATA. The binary search algorithm applied to our array DATA works as follows. During each stage of our algorithm, our search for ITEM is reduced to a segment of elements of DATA: $DATA[BEG]$, $DATA[BEG + 1]$, $DATA[BEG + 2]$, ..., $DATA[END]$. Note that the variables

BEG and END denote, respectively, the beginning and end locations of the segment under consideration.

The algorithm compares ITEM with the middle element DATA[MID] of the segment, where MID is obtained by $MID = \text{INT}((BEG + END)/2)$. (We use INT(A) for the integer value of A.) If DATA[MID] = ITEM, then the search is successful and we set LOC := MID. Otherwise a new segment of DATA is obtained as follows:

a) If $ITEM < DATA[MID]$, then ITEM can appear only in the left half of the segment: DATA[BEG), DATA [BEG + 1],, DATA[MID-1].

So, we reset $END := MID - 1$ and begin searching again.

b) If $ITEM > DATA[MID]$, then ITEM can appear only in the right half of the segment: DATA [MID+ 1], DATA [MID + 2]. DATA[END].

So, we reset $BEG := MID+1$ and begin searching again.

Initially, we begin with the entire array DATA: i.e., we begin with $BEG = 1$ and $END = n$, or, more generally, with $BEG = LB$ and $END = UB$. If ITEM is not in DATA, then eventually we obtain

$$END < BEG$$

This condition signals that the search is unsuccessful, and in such a case we assign $LOC := \text{NULL}$. Here NULL is a value that lies outside the set of indices of DATA. (In most cases, we can choose $\text{NULL} = 0$). We state the binary search algorithm formally.

Algorithm 4.6: (Binary Search) BINARY(DATA, LB, UB, ITEM, LOC)

Here DATA is a sorted array with lower bound LB and upper bound UB, and ITEM is a given item of information. The variables BEG, END and MID denote, respectively, the beginning, end and middle locations of a segment of elements of DATA. This algorithm finds the location LOC of ITEM in DATA or sets $LOC = \text{NULL}$.

1. [Initialize segment variables.]
Set $BEG := LB$, $END := UB$ and $MID = \text{INT}((BEG + END)/2)$.
2. Repeat Steps 3 and 4 while $BEG \leq END$ and $DATA[MID] \neq ITEM$.
3. If $ITEM < DATA[MID]$, then:
Set $END := MID - 1$.
Else:
Set $BEG := MID + 1$.
[End of If structure.]
4. Set $MID := \text{INT}((BEG + END)/2)$.
[End of Step 2 loop.]
5. If $DATA[MID] = ITEM$, then:
Set $LOC := MID$.
Else:
Set $LOC := \text{NULL}$.
[End of If structure.]
6. Exit.

Example 4.10

Let DATA be the following sorted 13-element array:

DATA: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

We apply the binary search to DATA for different values of ITEM.

- (a) Suppose ITEM = 40. The search for ITEM in the array DATA is pictured in Fig. 4.7, where the values of DATA[BEG] and DATA[END] in each stage of the algorithm are indicated by circles and the value of DATA[MID] by a square. Specifically, BEG, END and MID will have the following successive values:

1. Initially, BEG = 1 and END = 13. Hence

$$\text{MID} = \text{INT}[(1 + 13)/2] = 7 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 55$$

2. Since $40 < 55$, END has its value changed by $\text{END} = \text{MID} - 1 = 6$. Hence

$$\text{MID} = \text{INT}[(1 + 6)/2] = 3 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 30$$

3. Since $40 > 30$, BEG has its value changed by $\text{BEG} = \text{MID} + 1 = 4$. Hence

$$\text{MID} = \text{INT}[(4 + 6)/2] = 5 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 40$$

We have found ITEM in location $\text{LOC} = \text{MID} = 5$.

(1) (11), 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, (99)

(2) (11), 22, 30, 33, 40, (44), 55, 60, 66, 77, 80, 88, 99

(3) 11, 22, 30, 33, 40, (44), 55, 60, 66, 77, 80, 88, 99 [Successful]

Fig. 4.7 Binary Search for ITEM = 40

- (b) Suppose ITEM = 85. The binary search for ITEM is pictured in Fig. 4.8. Here BEG, END and MID will have the following successive values:

1. Again initially, BEG = 1, END = 13, MID = 7 and DATA[MID] = 55.

2. Since $85 > 55$, BEG has its value changed by $\text{BEG} = \text{MID} + 1 = 8$. Hence

$$\text{MID} = \text{INT}[(8 + 13)/2] = 10 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 77$$

3. Since $85 > 77$, BEG has its value changed by $\text{BEG} = \text{MID} + 1 = 11$. Hence

$$\text{MID} = \text{INT}[(11 + 13)/2] = 12 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 88$$

4. Since $85 < 88$, END has its value changed by $\text{END} = \text{MID} - 1 = 11$. Hence

$$\text{MID} = \text{INT}[(11 + 11)/2] = 11 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 80$$

(Observe that now $\text{BEG} = \text{END} = \text{MID} = 11$.)

Since $85 > 80$, BEG has its value changed by $\text{BEG} = \text{MID} + 1 = 12$. But now $\text{BEG} > \text{END}$. Hence ITEM does not belong to DATA.

(1) (11), 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, (99)

(2) 11, 22, 30, 33, 40, 44, 55, (60), 77, 80, 88, (99)

(3) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80), 88, (99)

(4) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80), 88, 99 [Unsuccessful]

Fig. 4.8 Binary Search for ITEM = 85

UNIT-II**Chapter-2**
Dynamic Memory Management**2.1 Introduction**

There are situations where the data is dynamic in nature. The number of data items can change when the program is being executed. For example, consider a program for processing the list of employees in an organization. This list can become longer as new employee names are added or shorter as employee names are deleted. When the list grows, more space has to be allocated to accommodate the additional data items. To handle such situations when they arise, dynamic data structures are used.

Before the execution of any program, all the static structures can have an allocation of fixed memory space for its own use. In the case of dynamic structures, the amount of memory required is not known in advance and thus cannot be allocated in this way. We can allocate the memory for the storage of the data items at runtime. Once our work is over, we can deallocate the memory. In C, there are two ways in which memory can be allocated:

1. by declaring variables
2. by explicitly requesting space from C

2.2 Memory Allocation and De-allocation functions

There are many functions in C used for allocating and managing memory:

- a) Malloc
 - b) Calloc
 - c) Realloc
 - d) Free
-
- a. **Malloc()** function may be used to allocate a block of memory. A block of memory with a particular size is reserved and a pointer is returned which is of type void meaning that this function can be assigned to any type of pointers.
 - b. **Calloc()** function is used when there is a need for multiple blocks of storage, of same size, and then set all bytes to zero
 - c. **Realloc()** function is used to move a reserved block of memory to another allocation of different dimensions. When the memory allocated by using calloc or malloc is insufficient or in excess, the size of the memory which has already been allocated can be changed using this function.

- d. **Free()** is used to release the previously used block of memory. In a block of memory when there is no further use of the data stored in it and also the same block is not used for the purpose of storing any other information, using this function the block of memory can be freed for using in the future.

Allocation a Block of Memory: malloc

A block of memory or single block of memory be allocated using the function malloc. The malloc reserves a block of memory of specified size and returns a pointer of type void. It takes the following general form:

```
ptr=(cast-type *)malloc(byte-size);
```

ptr is a pointer of type cast-type. The malloc returns a pointer (of cast-type) to an area of memory with size byte-size.

Example

```
x=(int *)malloc(100*size(int));
```

Equivalent to “100 times the size of an int”. Address of the first byte of the memory is assigned to the pointer x of the type int.

Allocation a Block of Memory : calloc

Calloc is another memory allocation function that is used for requesting memory space at run time for storing derived data types such as arrays and structures. Calloc allocates multiple blocks of storage, each of the same size, and then set all bytes to zero. General form:

```
ptr=(cast-type *)calloc(n, elem-size);
```

Allocates contiguous n blocks, each of size elem-size bytes. All bytes are initialized to zero and ptr points to the first byte of the allocated memory space. If there is not enough space, a NULL pointer is returned.

Altering the size of a block: realloc

The previously allocated memory is not sufficient and we need additional space for more elements. It is also possible that the memory allocated is much larger than necessary and we want to reduce it. In both cases, we can change the memory size already allocated with the help of the function realloc. The original allocation done by statement

```
ptr=malloc(size);
```

The reallocation of space done by statement

```
ptr=realloc(ptr, newsize);
```

Releasing the used space : free

When we no longer need the data we stored in a block of memory, and we do not intend to use that block for storing any other information, we may release that block of memory for future use, using the free function.

free(ptr)

ptr is a pointer to a memory block, which has already been created by malloc or calloc.

Example:

```
#include<string.h>
#include<stdlib.h>
#include<stdio.h>
#define NULL 0
void main()
{
    char *buffer;
    /*Allocating Memory*/
    if((buffer=(char*)malloc(10))!=NULL)
    {
        printf("Malloc Failed.\n");
        exit(1);
    }
    printf("Buffer of size %d is created\n",_msize(buffer));
    strcpy(buffer,"HYDERABAD");
    /*Re-Allocation*/
    if((buffer=(char*)realloc(buffer,15))!=NULL)
    {
        printf("Reallocation failed.\n");
        exit(1);
    }
    printf("\nBuffer size is modified.");
    printf("\nBuffer still contains: %s\n",buffer);
    strcpy(buffer,"SECUNDERABAD");
    printf("\nBuffer now contains: %s\n",buffer);
    /*Freeing memory*/
    free(buffer);
}
```

OUTPUT:

Buffer of size 10 is created

Buffer contains: HYDERABAD

Buffer size modified.

Buffer still contains: HYDERABAD

Buffer now contains: SECUNDERABAD

UNIT-II

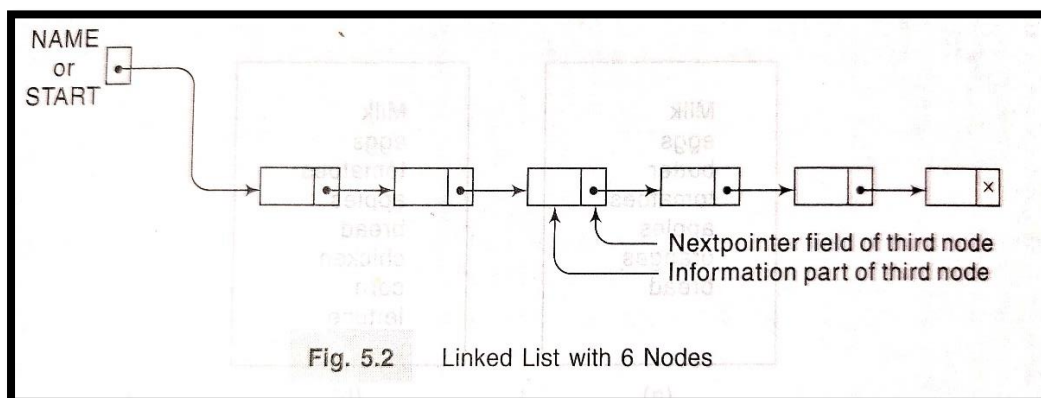
Chapter-3

Linked List

3.1 Linked List/Singly Liked List/One way List

A *linked list*, or *one-way list*, is a linear collection of data elements, called *nodes*, where the linear order is given by means of *pointers*. That is, each node is divided into two parts: the first part contains the information of the element, and the second part, called the *link field* or *nextpointer field* contains the address of the next node in the list.

Figure 5.2 is a schematic diagram of a linked list with 6 nodes. Each node is pictured with two parts. The left part represents the information part of the node, which may contain an entire record of data items (e.g., NAME, ADDRESS,...). The right part represents the nextpointer field of the node, and there is an arrow drawn from it to the next node in the list. The pointer of the last node contains a special value, called the null pointer, which is any invalid address.



The null pointer, denoted by x in the diagram, signals the end of the list. The linked list also contains a *list pointer variable* - called START or NAME-which contains the address of the first node in the list; hence There is an arrow drawn from START to the first node. Clearly, we need only this address in START to trace through the list. A special case is the list that has no nodes. Such a list is called the *null list* or *empty list* and is denoted by the null pointer in the variable START.

Example 5.1

A hospital ward contains 12 beds, of which 9 are occupied as shown in Fig. 5.3. Suppose we want an alphabetical listing of the patients. This listing may be given by the pointer field, called Next in the figure. We use the variable START to point to the first patient. Hence START contains 5, since the first patient, Adams, occupies bed 5. Also, Adams's pointer is equal to 3, since Dean, the next patient, occupies bed 3; Dean's pointer is 11, since Fields, the next patient, occupies bed 11; and so on. The entry for the last patient (Samuels) contains the null pointer, denoted by 0. (Some arrows have been drawn to indicate the listing of the first few patients.)

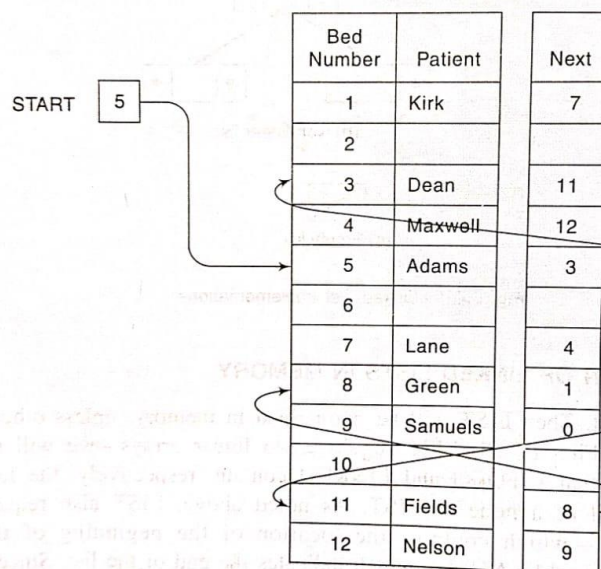


Fig. 5.3

3.2 Representation of Linked Lists in Memory

Let LIST be a linked list. Then LIST will be maintained in memory, unless otherwise specified or implied, as follows. First of all, LIST requires two linear arrays—we will call them here INFO and LINK—such that INFO[K] and LINK[K] contain, respectively, the information part and the nextpointer field of a node of LIST. LIST also requires a variable name—such as START—which contains the location of the beginning of the list, and a nextpointer sentinel—denoted by NULL—which indicates the end of the list. Since the subscripts of the arrays INFO and LINK will usually be positive, we will choose NULL = 0, unless otherwise stated.

The following examples of linked lists indicate that the nodes of a list need not occupy adjacent elements in the arrays INFO and LINK, and that more than one list may be maintained in the same linear arrays INFO and LINK. However, each list must have its own pointer variable giving the location of its first node.

Example 5.2

Figure 5.5 pictures a linked list in memory where each node of the list contains a single character. We can obtain the actual list of characters, or, in other words, the string, as follows:

START = 9, so INFO[9] = N is the first character.

LINK[9] = 3, so INFO[3] = O is the second character.

LINK[3] = 6, so INFO[6] = □ (blank) is the third character.

LINK[6] = 11, so INFO[11] = E is the fourth character.

LINK[11] = 7, so INFO[7] = X is the fifth character.

LINK[7] = 10, so INFO[10] = I is the sixth character.

LINK[10] = 4, so INFO[4] = T is the seventh character.

LINK[4] = 0, the NULL value, so the list has ended.

In other words, NO EXIT is the character string.

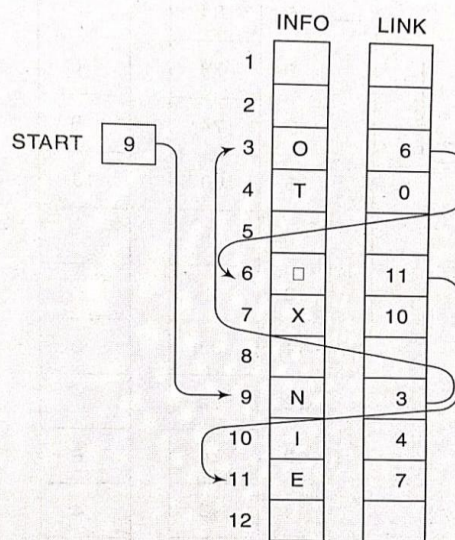
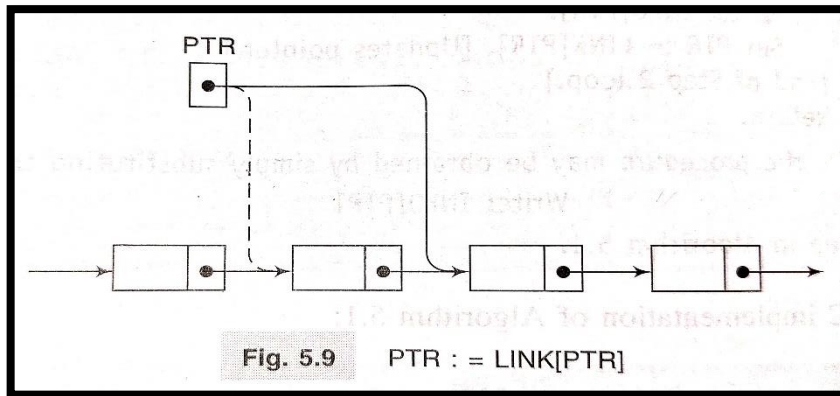


Fig. 5.5

3.3 Traversing a Linked List

Let LIST be a linked list in memory stored in linear arrays INFO and LINK with START pointing to the first element and NULL indicating the end of LIST. Suppose we want to traverse LIST in order to process each node exactly once. Traversing algorithm uses a pointer variable PTR which points to the node that is currently being processed. Accordingly, LINK[PTR] points to the next node to be processed. Thus the following assignment moves the pointer to the next node in the list

PTR := LINK[PTR]



Algorithm 5.1: (Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
- [End of Step 2 loop.]
5. Exit. //

Searching a Linked List

There are two searching algorithms for finding the location LOC of the node where ITEM first appears in LIST. The first algorithm does not assume that the data in LIST are sorted. The second algorithm does assume that LIST is sorted.

Searching: List is Unsorted

Algorithm 5.2 SEARCH(INFO, LINK, START, ITEM, LOC)
 LIST is a linked list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM = INFO[PTR], then:
 Set LOC := PTR, and Exit.
- Else:
 Set PTR := LINK[PTR]. [PTR now points to the next node.]
- [End of If structure.]
- [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit.

Searching: List is Sorted

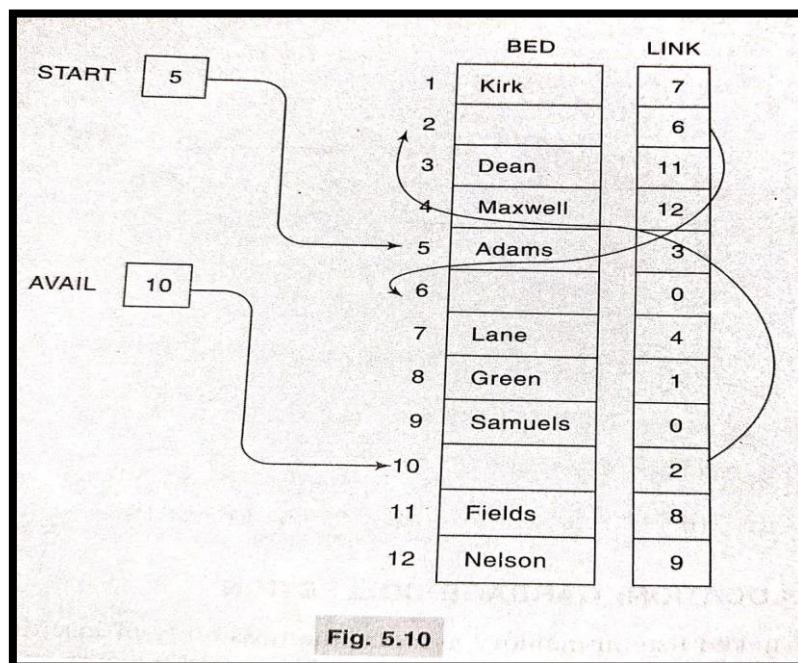
Algorithm 5.3: SRCHSL(INFO, LINK, START, ITEM, LOC)

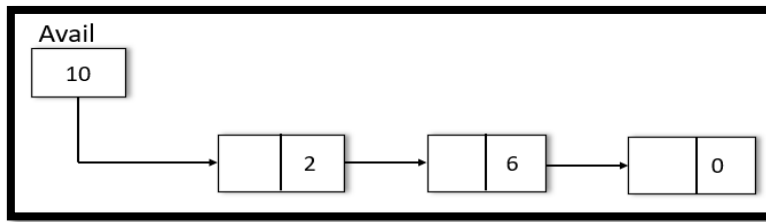
LIST is a sorted list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM > INFO[PTR], then:
 Set PTR := LINK[PTR]. [PTR now points to next node.]
 Else if ITEM = INFO[PTR], then:
 Set LOC := PTR, and Exit. [Search is successful.]
 Else:
 Set LOC := NULL, and Exit. [ITEM now exceeds INFO[PTR].]
 [End of If structure.]
 [End of Step 2 loop.]
4. Set LOC := NULL.
5. Exit.

3.4 Memory allocation: Garbage Collection

Program elements like variable, array, object are stored in the memory in the form of memory cells. In addition to the linked list which are stored in memory. Here, another special list is maintained which contains unused memory cell. This list is called list of available space or free storage list or free pool. The pointer of such a list is represented using a variable called AVAIL. Memory allocation means allocating or reserving memory for program elements such as arrays, linked list.





Garbage Collection means deleting unused memory cells used by program elements like variables, objects, arrays and linked list. The operating system of a computer may periodically collect all the deleted space onto the free- storage list. Any technique which does this collection is called **garbage collection**.

Garbage collection usually takes place in two steps.

1. First, the computer runs through all lists, tagging those cells which are currently in use.
2. Second, the computer runs through the memory, collecting all untagged space onto the free-storage list.

The garbage collection may take place when there is only some minimum amount of space or no space at all left in the free-storage list, or when the CPU is idle and has time to do the collection.

Overflow and Underflow

The situation where new data are to be inserted into a data structure but there is no available space, i.e., the free storage list is empty is called **overflow**. The overflow will occur in linked lists when $AVAIL = NULL$ and there is an insertion. The term **underflow** refers to the situation where one wants to delete data from a data structure that is empty. The underflow will occur in linked lists when $START = NULL$ and there is a deletion.

3.5 Inserting into linked List

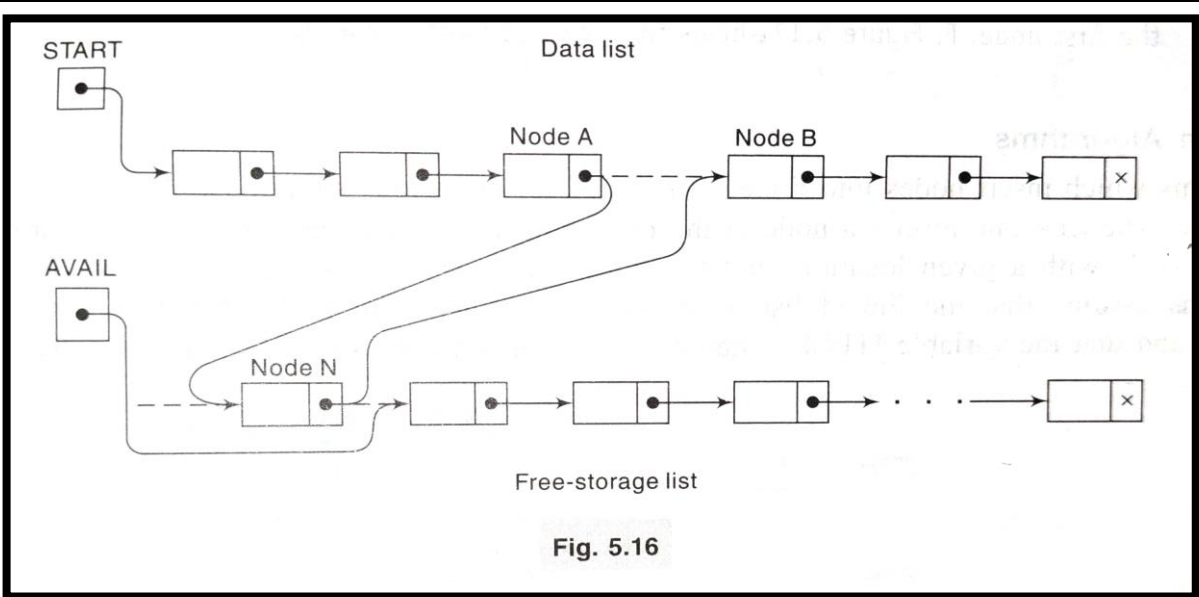
Let LIST be a linked list with successive nodes A and B. Suppose a node N is to be inserted into the list between nodes A and B. Need to take the memory space for the new node N from the AVAIL list. First node in the AVAIL list will be used for the new node N.

In figure 5.16 the pointer fields are changed as follows:

1. The **nextpointer** field of **node A** points to the new node N, to which AVAIL previously pointed.
2. **AVAIL** pointer points to the second node in the free pool, to which node N previously pointed.
3. The **nextpointer** field of **node N** now points to node B, to which node A previously pointed.

There are also two special cases.

- If the new node N is the first node in the list, then START will point to N;
- If the new node N is the last node in the list, then N will contain the null pointer.



Insertion algorithms will use a node in the **AVAIL** list, the algorithms will include the following steps:

- Checking to see if space is available in the AVAIL list. If not, that is, if AVAIL = NULL, then the algorithm will print the message OVERFLOW.
- Removing the first node from the AVAIL list. Using the variable NEW to keep track of the location of the new node, this step can be implemented by the pair of assignments
 $NEW := AVAIL$, $AVAIL := LINK[AVAIL]$
- Copying new information into the new node. In other words
 $INFO[NEW] := ITEM$

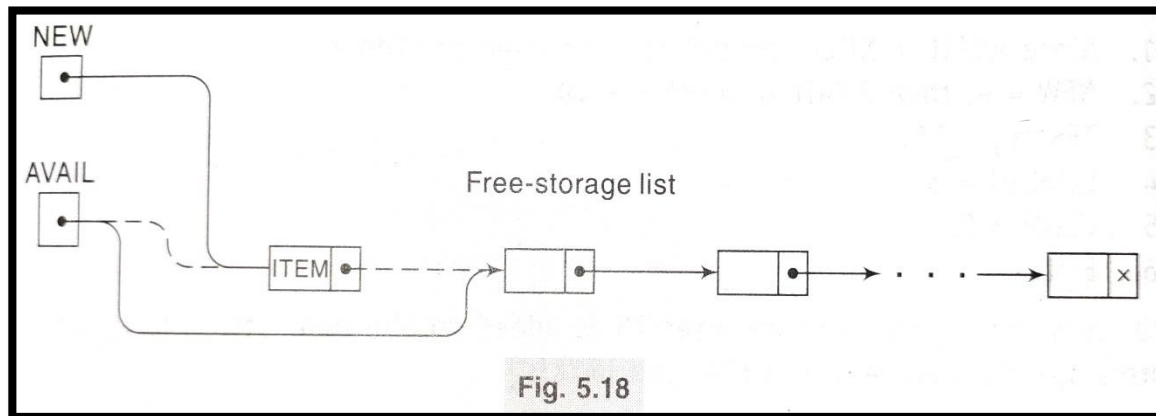
Inserting at the beginning of a List

Algorithm 5.4: $INSFIRST(INFO, LINK, START, AVAIL, ITEM)$

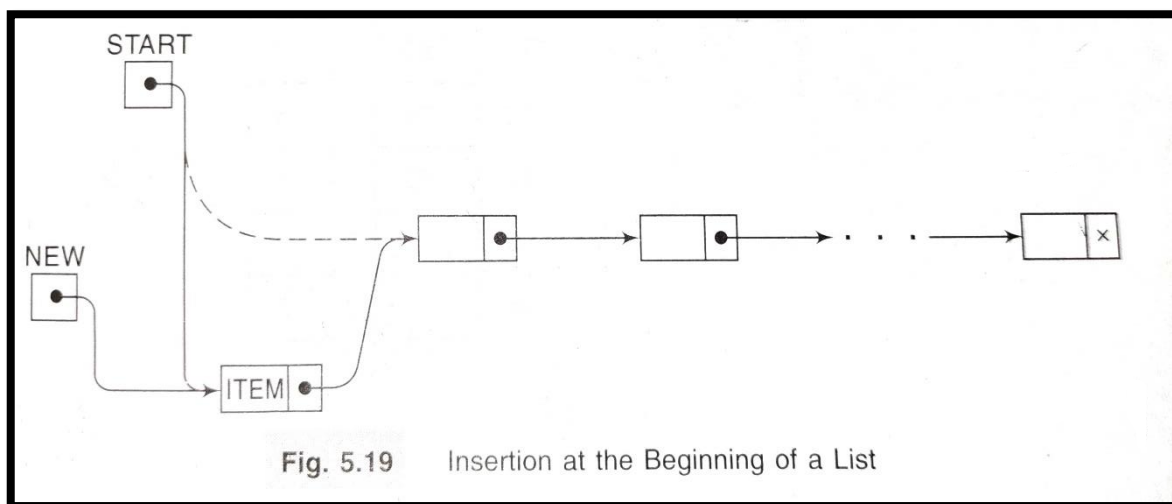
This algorithm inserts ITEM as the first node in the list.

- [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
- [Remove first node from AVAIL list.]
Set $NEW := AVAIL$ and $AVAIL := LINK[AVAIL]$.
- Set $INFO[NEW] := ITEM$. [Copies new data into new node]
- Set $LINK[NEW] := START$. [New node now points to original first node.]
- Set $START := NEW$. [Changes START so it points to the new node.]
- Exit.

Schematic Diagram of step 2 and 3



Schematic Diagram of step 4 and 5



Inserting after a given node

Algorithm 5.5: INSLOC(INFO, LINK, START, AVAIL, LOC, ITEM)

This algorithm inserts ITEM so that ITEM follows the node with location LOC or inserts ITEM as the first node when LOC = NULL.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM. [Copies new data into new node.]
4. If LOC = NULL, then: [Insert as first node.]
Set LINK[NEW] := START and START := NEW.
Else: [Insert after node with location LOC.]
Set LINK [NEW] := LINK[LOC] and LINK[LOC] := NEW.
[End of If structure.]
5. Exit.

Inserting at the end of the list

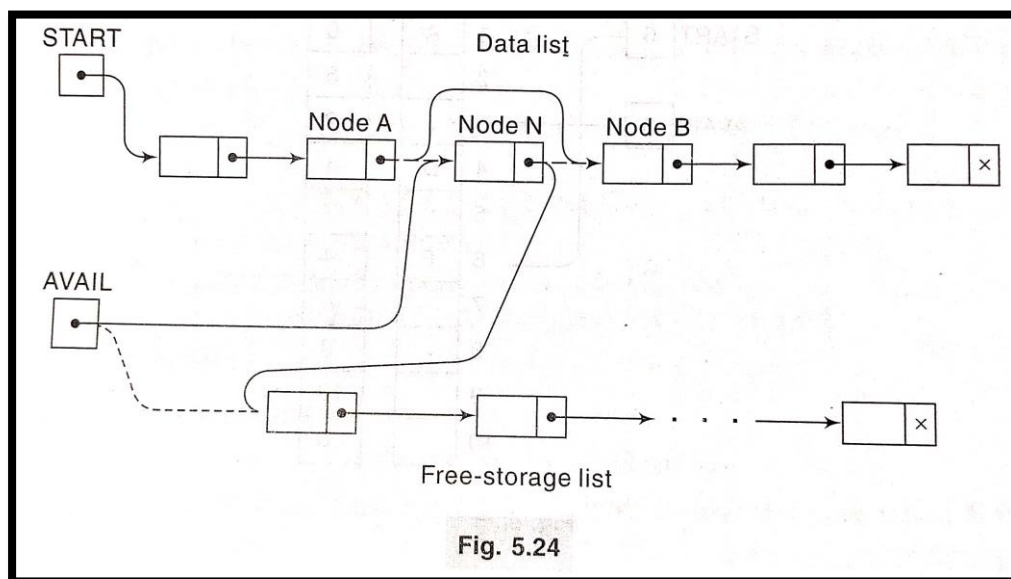
Algorithm : INSERTEND(INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM as the END node in the list.

1. If AVAIL=NULL, then: write OVERFLOW and Exit.
2. Set NEW:=AVAIL and AVAIL:=LINK[AVAIL].
3. Set INFO[NEW]:=ITEM
4. Set PTR:=START.
5. Repeat step 6 while LINK[PTR] $\neq$ NULL
Set PTR:=LINK[PTR].
[End of loop]
6. Set LINK[PTR]:=NEW and LINK[NEW]:=NULL.
7. Exit

3.6 Deletion from a Linked List

Let LIST be a linked list with a node N between nodes A and B. Suppose node N is to be deleted from the linked list. The deletion occurs as soon as the nextpointer field of node A is changed so that it points to node B. When a node N is deleted from our list, we will immediately return its memory space to the AVAIL-list.



The three pointer fields are changed as follows:

1. The nextpointer field of node A now points to node B, where node N previously pointed.
2. The nextpointer field of N now points to the original first node in free pool, where AVAIL previously pointed.
3. AVAIL now points to the deleted node N.

There are also two special cases.

- If the deleted node N is the first node in the list. then START will point to node B;

- If the deleted node N is the last node in the list, then node A will contain the NULL pointer.

Deletion algorithm will return the memory space of the deleted node N to the beginning of the AVAIL list. The deletion algorithm must check whether the node is present in the list or not. If not, that is $START = NULL$, print the message UNDERFLOW.

Deleting the first node from list

Algorithm: DELETEDFIRST(INFO, LINK, START, AVAIL, LOC)

This algorithm delete first node from the Linked List

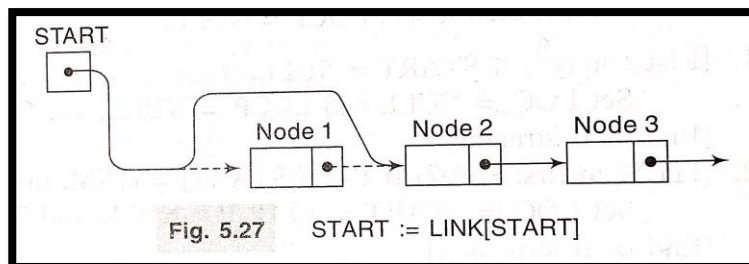
1. If $START = NULL$, then:
Write UNDERFLOW and Exit.
2. Set $LOC := START$ and $START := LINK[START]$.
3. Set $LINK[LOC] := AVAIL$ and $AVAIL := LOC$
4. Exit.

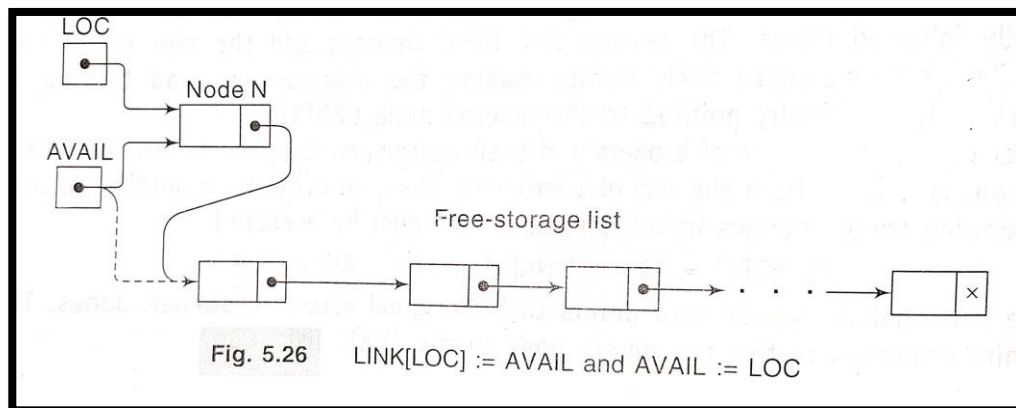
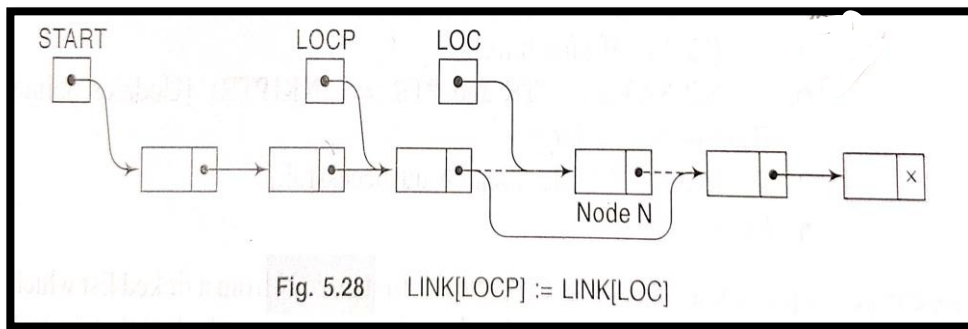
Deleting the Node Following a Given Node

Algorithm 5.8: DEL(INFO, LINK, START, AVAIL, LOC, LOCP)

This algorithm deletes the node N with location LOC. LOCP is the location of the node which precedes N or, when N is the first node, $LOCP = NULL$.

1. If $LOCP = NULL$, then:
Set $START := LINK[START]$. [Deletes first node.]
Else:
Set $LINK[LOCP] := LINK[LOC]$. [Deletes node N.]
[End of If structure.]
2. [Return deleted node to the AVAIL list.]
Set $LINK[LOC] := AVAIL$ and $AVAIL := LOC$.
3. Exit.





3.7 Types of Linked List

- Singly Linked List or one-way List
- Doubly Linked List or two-way List
- Circular Linked List
- Header Linked List

1. Singly Linked List

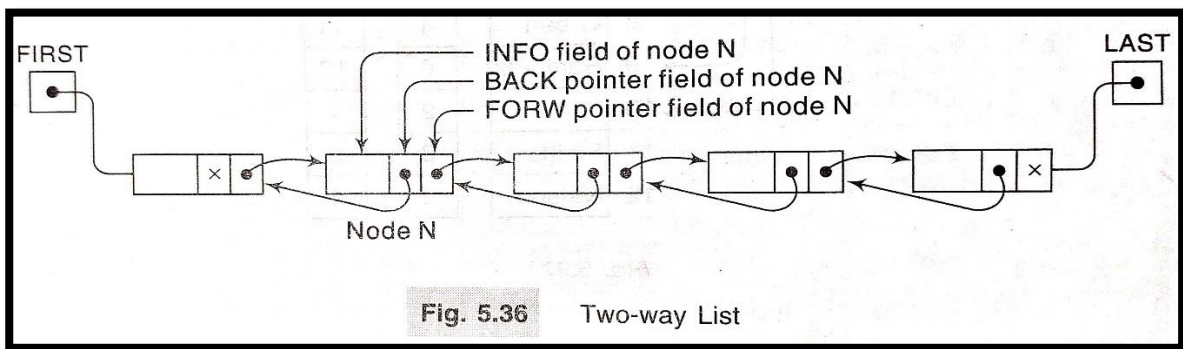
Already Discussed

2. Doubly Linked List

A two-way list is a linear collection of data elements, called nodes, where each node N is divided into three parts:

1. An information field INFO which contains the data of N
2. A pointer field FORW which contains the location of the next node in the list
3. A pointer field BACK which contains the location of the preceding node in the list.

The list also requires two list pointer variables: FIRST, which points to the first node in the list, and LAST, which points to the last node in the list.



A two-way list, which can be traversed in two directions:

- In a forward direction from the beginning of the list to the end.
- In the backward direction from the end of the list to the beginning.

Given the location LOC of a node N in the list, one now has immediate access to both the next node and the preceding node in the list. This means, in particular, that one is able to delete N from the list without traversing any part of the list.

The null pointer appears in the FORW field of the last node in the list and also in the BACK field of the first node in the list. Using the variable FIRST and the pointer field FORW, we can traverse a two-way list in the forward direction. On the other hand, using the variable LAST and the pointer field BACK, we can also traverse the list in the backward direction. Suppose LOCA and LOCB are the locations, respectively, of nodes A and B in a two-way list. Then the way that the pointers FORW and BACK are defined gives us the following:

Pointer property:

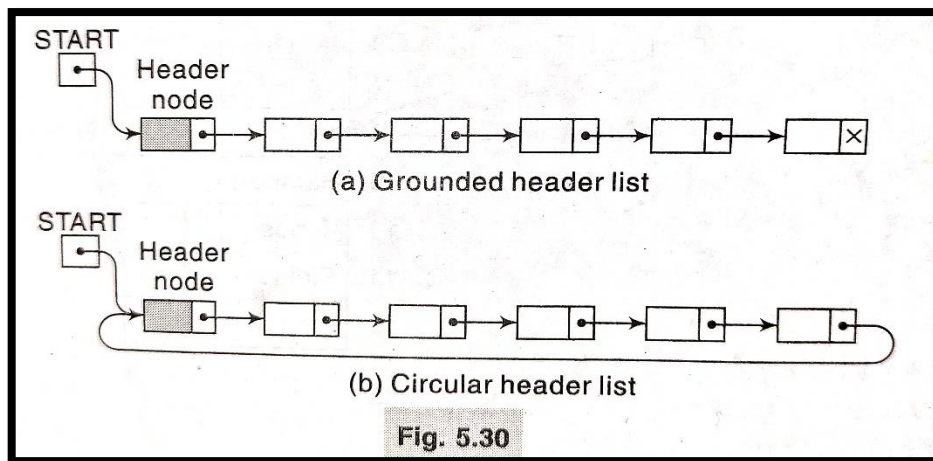
$$\text{FORW}[\text{LOCA}] = \text{LOCB} \text{ if and only if } \text{BACK}[\text{LOCB}] = \text{LOCA}$$

The statement that node B follows node A is equivalent to the statement that node A precedes node B. Two-way lists may be maintained in memory by means of linear arrays in the same way as one-way lists except that two pointer arrays, FORW and BACK, instead of one pointer array LINK, and we require two list pointer variables, FIRST and LAST, instead of one list pointer variable START. The list AVAIL of available space in the arrays will still be maintained as a one-way list-using FORW as the pointer field-since we delete and insert nodes only at the beginning of the AVAIL list.

3. Header Linked List

A header-linked list is a linked list that always contains a special node, called the header node, at the beginning of the list. The following are two kinds of header lists:

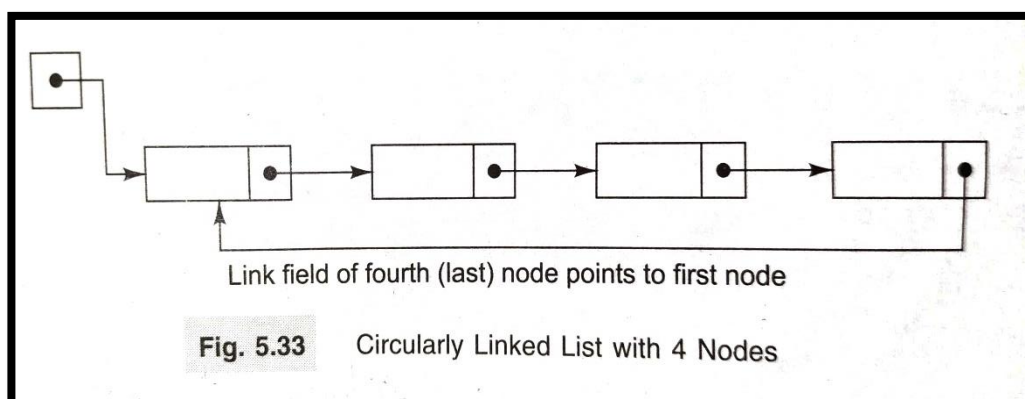
1. A grounded header list is a header list where the last node contains the null pointer.
2. A circular header list is a header list where the last node points back to the header node.



Unless otherwise stated, header lists will always be circular. Observe that the list pointer START always points to the header node. Accordingly, $\text{LINK}[\text{START}] = \text{NULL}$ indicates that a grounded header list is empty, and $\text{LINK}[\text{START}] = \text{START}$ indicates that a circular header list is empty. Data may be maintained by header lists in memory, the AVAIL list will always be maintained as an ordinary linked list.

3. Circular Linked List

Circularly linked list is a special type of linked list. In a circularly linked list, the link field of the last node points to the first node of the list. This type of list is mainly used in lists that allow access to nodes in the middle of the list without starting at the beginning. The last node does not contain a NULL pointer as in a singly linked list. The link field of the last node is connected to the information part of the first node.



Example 5.22

Consider the circularly linked list in Fig. 5.34. It has five nodes. Notice that the last node is not a NULL node. It contains an element 30. The pointer in the last node points back to the first node, which contains the element 10.

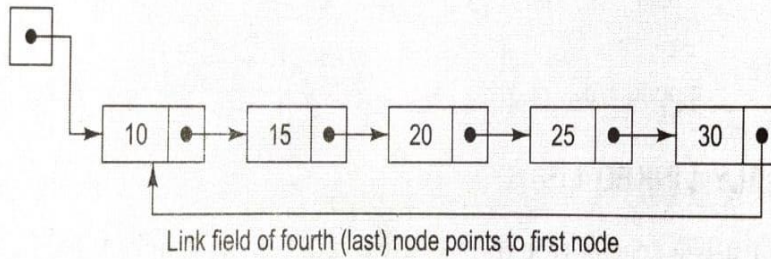


Fig. 5.34 Example of a circularly Linked List